

Semana 3 - Redes Neurais

João Florindo

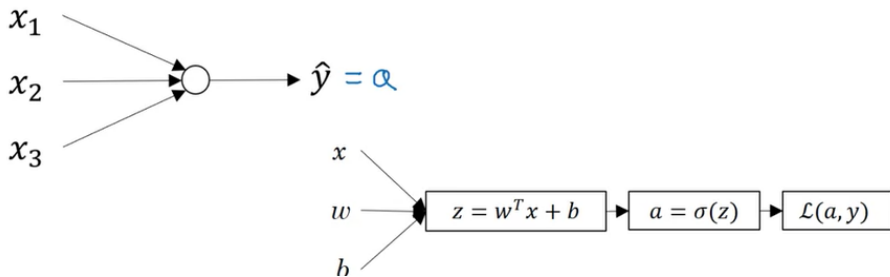
Instituto de Matemática, Estatística e Computação Científica
Universidade Estadual de Campinas - Brasil
florindo@unicamp.br

Outline

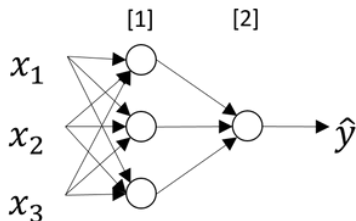
1 Redes Neurais

2 Gradiente descendente

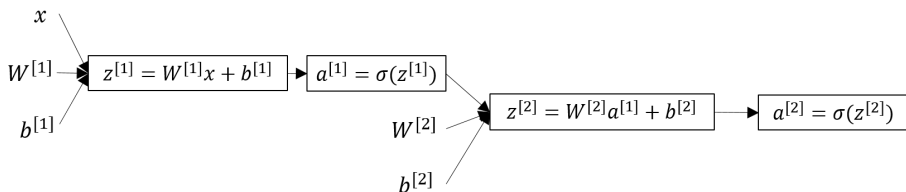
Na Regressão Logística:



Uma rede de duas camadas:



Grafo computacional:



- Três tipos de camadas: entrada, escondida e saída:
 - Entrada: *features* $x_1, x_2, \dots$.
 - Escondida: valores não observados no conjunto de treino.
 - Saída: previsão do modelo.
- A entrada não é contada no número de camadas.
- Treinar uma rede significa obter os parâmetros W e b associados com as camadas escondida e de saída.

- Três tipos de camadas: entrada, escondida e saída:
 - Entrada: *features* $x_1, x_2, \dots$.
 - Escondida: valores não observados no conjunto de treino.
 - Saída: previsão do modelo.
- A entrada não é contada no número de camadas.
- Treinar uma rede significa obter os parâmetros W e b associados com as camadas escondida e de saída.

- Três tipos de camadas: entrada, escondida e saída:
 - Entrada: *features* $x_1, x_2, \dots$.
 - Escondida: valores não observados no conjunto de treino.
 - Saída: previsão do modelo.
- A entrada não é contada no número de camadas.
- Treinar uma rede significa obter os parâmetros W e b associados com as camadas escondida e de saída.

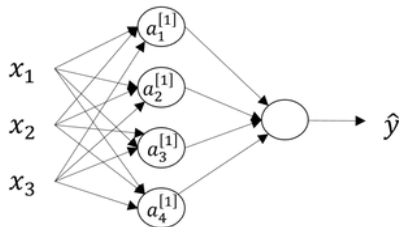
- Três tipos de camadas: entrada, escondida e saída:
 - Entrada: *features* $x_1, x_2, \dots$.
 - Escondida: valores não observados no conjunto de treino.
 - Saída: previsão do modelo.
- A entrada não é contada no número de camadas.
- Treinar uma rede significa obter os parâmetros W e b associados com as camadas escondida e de saída.

- Três tipos de camadas: entrada, escondida e saída:
 - Entrada: *features* $x_1, x_2, \dots$.
 - Escondida: valores não observados no conjunto de treino.
 - Saída: previsão do modelo.
- A entrada não é contada no número de camadas.
- Treinar uma rede significa obter os parâmetros W e b associados com as camadas escondida e de saída.

- Três tipos de camadas: entrada, escondida e saída:
 - Entrada: *features* $x_1, x_2, \dots$.
 - Escondida: valores não observados no conjunto de treino.
 - Saída: previsão do modelo.
- A entrada não é contada no número de camadas.
- Treinar uma rede significa obter os parâmetros W e b associados com as camadas escondida e de saída.

Forward

Cálculo da saída:



$$z_1^{[1]} = w_1^{[1]T} x + b_1^{[1]}, a_1^{[1]} = \sigma(z_1^{[1]})$$

$$z_2^{[1]} = w_2^{[1]T} x + b_2^{[1]}, a_2^{[1]} = \sigma(z_2^{[1]})$$

$$z_3^{[1]} = w_3^{[1]T} x + b_3^{[1]}, a_3^{[1]} = \sigma(z_3^{[1]})$$

$$z_4^{[1]} = w_4^{[1]T} x + b_4^{[1]}, a_4^{[1]} = \sigma(z_4^{[1]})$$

Vetorizando:

$$z^{[1]} = \begin{bmatrix} z_1^{[1]} \\ z_2^{[1]} \\ z_3^{[1]} \\ z_4^{[1]} \end{bmatrix} \quad W^{[1]} = \begin{bmatrix} \text{---}(w_1^{[1]})^T\text{---} \\ \text{---}(w_2^{[1]})^T\text{---} \\ \text{---}(w_3^{[1]})^T\text{---} \\ \text{---}(w_4^{[1]})^T\text{---} \end{bmatrix}$$

$$z^{[2]} = \begin{bmatrix} z_1^{[2]} \end{bmatrix} \quad W^{[2]} = \begin{bmatrix} \text{---}(w_1^{[2]})^T\text{---} \end{bmatrix}$$

Forward

$$z^{[1]} = W^{[1]}x + b^{[1]}$$

$$a^{[1]} = \sigma(z^{[1]})$$

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

$$a^{[2]} = \sigma(z^{[2]})$$

- $z^{[1]}$ é a combinação linear
- $a^{[1]}$ é a ativação (sigmoide de $z^{[1]}$)
- n_x é o número de *features* na entrada x (acima $n_x = 3$)
- n_1 é o número de neurônios na camada escondida (acima $n_1 = 4$)

Dimensões

Variável	Dimensão	Descrição
x	$(n_x, 1)$	Vetor de entradas com n_x <i>features</i>
$W^{[1]}$	(n_1, n_x)	Matriz de pesos da primeira camada (escondida)
$b^{[1]}$	$(n_1, 1)$	<i>Biases</i> da camada escondida
$z^{[1]}$	$(n_1, 1)$	Combinação linear da camada escondida
$a^{[1]}$	$(n_1, 1)$	Ativação da camada escondida
$W^{[2]}$	$(1, n_1)$	Matriz de pesos da segunda camada (saída)
$b^{[2]}$	$(1, 1)$	<i>Biases</i> da camada de saída
$z^{[2]}$	$(1, 1)$	Combinação linear da camada de saída
$a^{[2]}$	$(1, 1)$	Ativação da camada de saída, i.e., o valor de saída

Código

Precisamos calcular $z[1]$, $a[1]$, $z[2]$, $a[2]$ para cada um dos m exemplos:

```
for i in range(m):  
    z[1][i] = W[1]*x[i] + b[1]  
    a[1][i] = sigmoid(z[1][i])  
    z[2][i] = W[2]*a[1][i] + b[2]  
    a[2][i] = sigmoid(z[2][i])
```

Vetorização por Exemplo

Empilhamos a entrada x e as ativações de cada exemplo em colunas:

$$X = \begin{bmatrix} | & | & & | \\ x^{(1)} & x^{(2)} & \dots & x^{(m)} \\ | & | & & | \end{bmatrix}$$

$$A^{[1]} = \begin{bmatrix} | & | & & | \\ a^{[1]}(1) & a^{[1]}(2) & \dots & a^{[1]}(m) \\ | & | & & | \end{bmatrix}$$

Vetorização por Exemplo

Algumas variáveis são redefinidas:

Variável	Dimensão	Descrição
X	(n_x, m)	Matriz de m entradas com n_x <i>features</i>
$W^{[1]}$	(n_1, n_x)	Matriz de pesos da primeira camada (escondida)
$b^{[1]}$	$(n_1, 1)$	<i>Biases</i> da camada escondida
$Z^{[1]}$	(n_1, m)	Combinação linear da camada escondida para os exemplos
$A^{[1]}$	(n_1, m)	Ativação da camada escondida
$W^{[2]}$	$(1, n_1)$	Matriz de pesos da segunda camada (saída)
$b^{[2]}$	$(1, 1)$	<i>Biases</i> da camada de saída
$Z^{[2]}$	$(1, m)$	Combinação linear da camada de saída
$A^{[2]}$	$(1, m)$	Ativação da camada de saída, i.e., o valor de saída

Vetorização por Exemplo

Agora tudo é calculado de uma vez:

$$Z^{[1]} = W^{[1]}X + b^{[1]}$$

$$A^{[1]} = \sigma(Z^{[1]})$$

$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$$

$$A^{[2]} = \sigma(Z^{[2]})$$

Código:

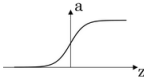
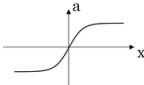
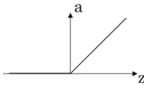
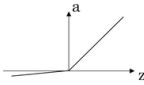
$$Z[1] = W[1]*X + b[1]$$

$$A[1] = \text{sigmoid}(Z[1])$$

$$Z[2] = W[2]*A[1] + b[2]$$

$$A[2] = \text{sigmoid}(Z[2])$$

Funções de Ativação

ativação	fórmula	gráfico	descrição
sigmoide	$a = \frac{1}{1 + e^{-z}}$		sigmoide logística, use se sua saída deve ter valores entre 0 e 1
tanh	$a = \frac{e^z - e^{-z}}{e^z + e^{-z}}$		costuma funcionar melhor que a sigmoide na camada escondida por centralizar a saída em torno de zero
ReLU	$a = \max(0, z)$		<i>rectified linear unit</i> , a função de ativação mais amplamente usada
Leaky ReLU	$a = \max(0, 0.01z, z)$		versão melhorada da ReLU, 0.01 pode ser um hiperparâmetro

Funções de Ativação

- Por que não linear: composição de funções lineares é sempre uma função linear (equivalente a uma regressão linear ou logística).
- Exceção pode ser na saída de uma regressão.
- Podemos ter ativações diferentes em cada camada i : no caso denotamos $g(z)^{[i]}$.
- Tanto a sigmoide quanto a tanh têm derivada muito próxima de zero para z grande pequeno.
- A solução para isso é a ReLU.

Funções de Ativação

- Por que não linear: composição de funções lineares é sempre uma função linear (equivalente a uma regressão linear ou logística).
- Exceção pode ser na saída de uma regressão.
- Podemos ter ativações diferentes em cada camada i : no caso denotamos $g(z)^{[i]}$.
- Tanto a sigmoide quanto a tanh têm derivada muito próxima de zero para z grande pequeno.
- A solução para isso é a ReLU.

Funções de Ativação

- Por que não linear: composição de funções lineares é sempre uma função linear (equivalente a uma regressão linear ou logística).
- Exceção pode ser na saída de uma regressão.
- Podemos ter ativações diferentes em cada camada i : no caso denotamos $g(z)^{[i]}$.
- Tanto a sigmoide quanto a tanh têm derivada muito próxima de zero para z grande pequeno.
- A solução para isso é a ReLU.

Funções de Ativação

- Por que não linear: composição de funções lineares é sempre uma função linear (equivalente a uma regressão linear ou logística).
- Exceção pode ser na saída de uma regressão.
- Podemos ter ativações diferentes em cada camada i : no caso denotamos $g(z)^{[i]}$.
- Tanto a sigmoide quanto a tanh têm derivada muito próxima de zero para z grande pequeno.
- A solução para isso é a ReLU.

Funções de Ativação

- Por que não linear: composição de funções lineares é sempre uma função linear (equivalente a uma regressão linear ou logística).
- Exceção pode ser na saída de uma regressão.
- Podemos ter ativações diferentes em cada camada i : no caso denotamos $g(z)^{[i]}$.
- Tanto a sigmoide quanto a tanh têm derivada muito próxima de zero para z grande pequeno.
- A solução para isso é a ReLU.

Funções de Ativação

Derivadas:

ativação	fórmula	derivada
sigmoid	$a = \frac{1}{1 + e^{-z}}$	$a(1-a)$
tanh	$a = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$1-a^2$
ReLU	$a = \max(0, z)$	0 if $z < 0$; 1 if $z \geq 0$
Leaky ReLU	$a = \max(0.01z, z)$	0.01 if $z < 0$; 1 if $z \geq 0$

NOTA: Derivada da ReLU e Leaky ReLU não é definida para $z=0$, porém, usa-se tanto 1 quanto 0 e funciona (**subgradiente**).

Outline

1 Redes Neurais

2 Gradiente descendente

Vamos generalizar agora n_2 para um valor qualquer, preparando para o próximo tópico.

Variável	Dimensão	Descrição
X	(n_x, m)	Matriz de m entradas com n_x <i>features</i>
$Z^{[1]}$	(n_1, m)	Combinação linear da camada escondida para os exemplos
$A^{[1]}$	(n_1, m)	Ativação da camada escondida
$Z^{[2]}$	$(n_2, 1)$	Combinação linear da camada de saída
$A^{[2]}$	$(n_2, 1)$	Ativação da camada de saída, i.e., o valor de saída
$W^{[1]}$	(n_1, n_x)	Matriz de pesos da primeira camada (escondida)
$b^{[1]}$	$(n_1, 1)$	<i>Biases</i> da camada escondida
$W^{[2]}$	(n_2, n_1)	Matriz de pesos da segunda camada (saída)
$b^{[2]}$	$(n_2, 1)$	<i>Biases</i> da camada de saída

Forward Propagation

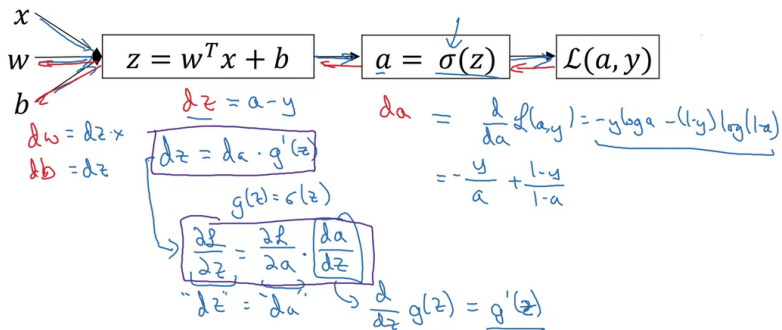
$$Z[1] = W[1]*X + b[1]$$

$$A[1] = \text{sigmoid}(Z[1])$$

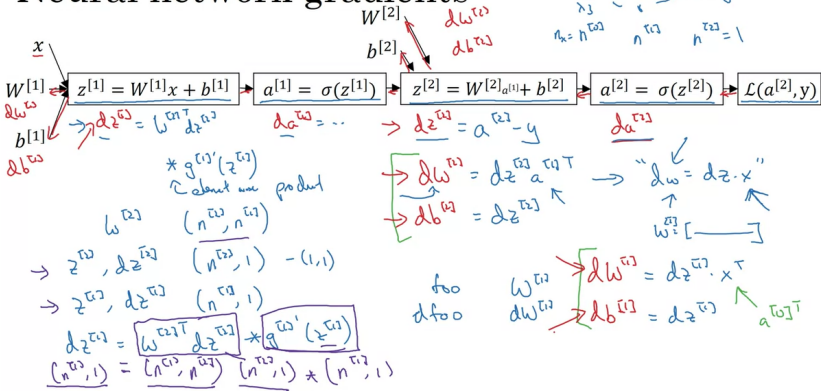
$$Z[2] = W[2]*A[1] + b[2]$$

$$A[2] = \text{sigmoid}(Z[2])$$

Logistic regression



Neural network gradients



Equações para Um Exemplo

$$dz^{[2]} = a^{[2]} - y$$

$$dW^{[2]} = dz^{[2]} a^{[1]T}$$

$$db^{[2]} = dz^{[2]}$$

$$dz^{[1]} = W^{[2]T} dz^{[2]} * g^{[1]'}(z^{[1]})$$

$$dW^{[1]} = dz^{[1]} x^T$$

$$db^{[1]} = dz^{[1]}$$

Vetorização

$$\underline{dz^{[2]}} = \underline{a^{[2]}} - \underline{y}$$

$$dW^{[2]} = dz^{[2]} a^{[1]T}$$

$$db^{[2]} = dz^{[2]}$$

$$\underbrace{dz^{[1]}}_{(n^{[1]}, 1)} = W^{[2]T} dz^{[2]} * g^{[1]'}(z^{[1]})$$

$$dW^{[1]} = dz^{[1]} x^T$$

$$db^{[1]} = dz^{[1]}$$

$$\underline{dZ^{[2]}} = \underline{A^{[2]}} - \underline{Y}$$

$$dW^{[2]} = \frac{1}{m} dZ^{[2]} A^{[1]T}$$

$$db^{[2]} = \frac{1}{m} np.sum(dZ^{[2]}, axis = 1, keepdims = True)$$

$$\underbrace{dZ^{[1]}}_{(n^{[1]}, m)} = \underbrace{W^{[2]T}}_{(n^{[2]}, m)} \underbrace{dZ^{[2]}}_{(n^{[2]}, m)} * \underbrace{g^{[1]'}(Z^{[1]})}_{(n^{[1]}, m)}$$

↙ elementwise product

$$dW^{[1]} = \frac{1}{m} dZ^{[1]} X^T$$

$$db^{[1]} = \frac{1}{m} np.sum(dZ^{[1]}, axis = 1, keepdims = True)$$

$$J(\dots) = \frac{1}{m} \sum_{i=1}^n \mathcal{L}(\hat{y}_i, y_i)$$

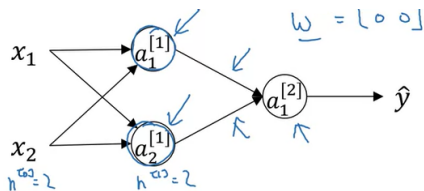
Backpropagation - Código

Repita até convergir:

```
# backpropagation
dZ[2] = A[2] - Y # obtida pela combinacao da derivada
                  # da funcao de custo e g'[2]
dW[2] = 1/m * np.matmul(dZ[2], A[1].T)
db[2] = 1/m * np.sum(dZ[2], axis=1, keepdims=True)
dZ[1] = np.multiply(np.matmul(W[2].T, dZ[2]), g'[1](Z[1]))
                  # derivada da ativacao usada aqui
dW[1] = 1/m * np.matmul(dZ[1], X.T)
db[1] = 1/m * np.sum(dZ[1])

# atualizar parametros
W[1] = W[1] - learning_rate * dW[1]
b[1] = b[1] - learning_rate * db[1]
W[2] = W[2] - learning_rate * dW[2]
b[2] = b[2] - learning_rate * db[2]
```


Inicialização



$$W_{\kappa}^{(1)} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

$$a_1^{(1)} = a_2^{(1)}$$

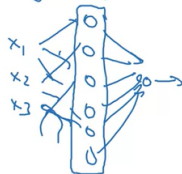
$$dW = \begin{bmatrix} u & v \\ u & v \end{bmatrix}$$

$$b_{\kappa}^{(1)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$dz_1^{(1)} = dz_2^{(1)}$$

$$W^{(1)} = W^{(1)} - \alpha dW$$

Symmetric



$$W^{(1)} = \begin{bmatrix} \dots & \dots \\ \dots & \dots \end{bmatrix}$$

Inicialização

```

W[1] = np.random.randn((n1, nx)) * 0.01  # numeros pequenos
                                           # aleatorios
b[1] = np.zeros((n1, 1))                  # zeros eh ok para os
                                           # termos de bias

W[2] = np.random.randn((n2, n1)) * 0.01
b[2] = np.zeros((n2, 1))

```

Por que aleatório?

- Quebrar simetria
- Se todos os pesos começarem iguais, todas as saídas da camada escondida sempre serão iguais

Por que números pequenos?

- Na sigmoide ou tanh, se w for grande, $z = wx + b$ será grande, e a derivada $g'(z) \approx 0$, tornando o aprendizado lento (0.01 pode ser alterado!)

Inicialização

```

W[1] = np.random.randn((n1, nx)) * 0.01  # numeros pequenos
                                           # aleatorios
b[1] = np.zeros((n1, 1))                  # zeros eh ok para os
                                           # termos de bias

W[2] = np.random.randn((n2, n1)) * 0.01
b[2] = np.zeros((n2, 1))

```

Por que aleatório?

- Quebrar simetria
- Se todos os pesos começarem iguais, todas as saídas da camada escondida sempre serão iguais

Por que números pequenos?

- Na sigmoide ou tanh, se w for grande, $z = wx + b$ será grande, e a derivada $g'(z) \approx 0$, tornando o aprendizado lento (0.01 pode ser alterado!)

Inicialização

```

W[1] = np.random.randn((n1, nx)) * 0.01  # numeros pequenos
                                           # aleatorios
b[1] = np.zeros((n1, 1))                  # zeros eh ok para os
                                           # termos de bias

W[2] = np.random.randn((n2, n1)) * 0.01
b[2] = np.zeros((n2, 1))

```

Por que aleatório?

- Quebrar simetria
- Se todos os pesos começarem iguais, todas as saídas da camada escondida sempre serão iguais

Por que números pequenos?

- Na sigmoide ou tanh, se w for grande, $z = wx + b$ será grande, e a derivada $g'(z) \approx 0$, tornando o aprendizado lento (0.01 pode ser alterado!)